ML4350 - Reinforcement Learning - Winter 2025/26
Final Project Report

ShaBin: Abtin Mogharabin, Kourosh Sharifi

February 26, 2026

1 Introduction

The focus of this project is the application of deep reinforcement learning within a competitive hockey environment. This environment is built upon the Gymnasium framework [10] and utilizes the Box2D physics engine to simulate the dynamics of a hockey game. The state space is composed of an 18-dimensional continuous vector representing the positions, velocities, and angles of the two players and the puck. The action space is four-dimensional and continuous, allowing for control over horizontal and vertical acceleration, rotational torque, and a discrete shooting mechanism.

The primary objective for the agent is to outperform a set of heuristic-based opponents provided by the environment. These include a "weak" opponent, which moves slowly and follows basic puck-tracking logic, and a "strong" opponent with higher speeds and more aggressive scoring heuristics. To facilitate learning, the environment provides three specialized modes: a shooting mode for offensive practice, a defense mode for goal-keeping, and a normal mode for full matches.

This report details the implementation and evaluation of two main off-policy algorithms:

- Truncated Quantile Critics (TQC) [7] by Abtin Mogharabin
- Soft Actor-Critic (SAC) [4] by Kourosh Sharifi

While other architectures such as Twin Delayed Deep Deterministic Policy Gradient (TD3) [3], REDQ [1], and DroQ [5] were initially implemented and tested, TQC and SAC demonstrated the most robust convergence properties in this high-variance continuous control task in combination to our self-play strategies. A core component of the methodology involves a multi-phase curriculum, potential-based reward shaping, and a final Policy-Space Response Oracles (PSRO) [8] league framework to ensure the agents generalize against a diverse set of strategies. The complete source code and implementation details are available in the project repository.[1]

2 Methods

2.1 Truncated Quantile Critics (TQC)

To address the issue of overestimation bias in off-policy continuous control, I (**Abtin Mogharabin**) implement Truncated Quantile Critics (TQC) [7]. TQC extends the Soft Actor-Critic (SAC) framework by

[1] Code available at: https://github.com/KouroshKSH/ML4350-RL-Hockey-Agent

integrating three core mechanisms: distributional representation of the return, truncation of the predicted distribution, and ensembling of multiple critics, with two 256-unit MLP hidden layers per network.

2.1.1 Implementation

In TQC, the standard scalar Q-function is replaced by a distributional return $Z^\pi(s,a)$. Following the Quantile Regression (QR) approach, the distribution for each critic $i \in \{1,\ldots,N\}$ is approximated as a uniform mixture of M Dirac delta functions: $Z_{\psi_i}(s,a) = \frac{1}{M}\sum_{j=1}^{M}\delta(\theta_{i,j}(s,a))$ where $\theta_{i,j}(s,a)$ represents the j-th quantile location (atom) for the i-th critic network, learned for quantile fractions $\tau_j = \frac{2j-1}{2M}$ for $j = 1,\ldots,M$.
To construct the temporal difference (TD) target, TQC first aggregates the atoms from all N target networks into a single mixture $\mathcal{Z}_{total} = \bigcup_{i=1}^{N}\{\theta'_{i,1},\ldots,\theta'_{i,M}\}$, where θ' denotes atoms from the target networks evaluated at (s',a') with $a' \sim \pi(\cdot|s')$. Overestimation is controlled by sorting these NM atoms in ascending order and dropping the k topmost values. The resulting truncated target distribution $\mathcal{T}$ is defined as $\mathcal{T} = \{\theta_{(1)},\theta_{(2)},\ldots,\theta_{(NM-k)}\}$ where $\theta_{(j)}$ is the j-th smallest atom in $\mathcal{Z}_{total}$. The target quantiles for the i-th critic are then computed using the Bellman operator $y_m = r + \gamma\left(\theta_{(m)} - \alpha\log\pi(a'|s')\right)$
The critics are updated by minimizing the averaged 1-Wasserstein distance, implemented via the Quantile Huber loss (with $\kappa = 1$):

$$J_Q(\psi_i) = \mathbb{E}_{\mathcal{D}}\left[\frac{1}{M}\sum_{j=1}^{M}\sum_{m=1}^{NM-k}\rho^H_{\tau_j}(y_m - \theta_{i,j}(s,a))\right] \tag{1}$$

where ρ^H_τ is the asymmetric Huber loss.
The actor policy π_ϕ is optimized to maximize both the expected return (calculated as the mean of all critic atoms) and the policy entropy $\mathcal{H}(\pi_\phi(\cdot|s))$, using a tanh-squashed Gaussian policy with log-std clamped to $[-20, 2]$:

$$J_\pi(\phi) = \mathbb{E}_{s\sim\mathcal{D},a\sim\pi_\phi}\left[\frac{1}{N}\sum_{i=1}^{N}\left(\frac{1}{M}\sum_{j=1}^{M}\theta_{i,j}(s,a)\right) + \alpha\mathcal{H}(\pi_\phi(\cdot|s))\right] \tag{2}$$

My specific implementation utilizes $N = 5$ critics and $M = 25$ quantiles per critic, resulting in a pool of 125 atoms. I apply a truncation parameter of $k = 2$ to provide a conservative yet high-fidelity estimation of the Q-values, and I train with Adam at 2×10^{-4}, batch size 256, discount $\gamma = 0.99$, and Polyak target updates with $\tau = 0.005$. The learning rate and noise were tuned specifically, as described in section 2.1.4 below.

2.1.2 Reward Shaping

To make learning less sparse, I added a compact shaping reward on top of the environment signals. I use the environment-provided proximity proxy `reward_closeness_to_puck` as a continuous incentive to approach the puck, and I include an opponent-possession penalty using the mirrored touch proxy `reward_touch_puck` (evaluated for the opponent) so that shoting the puck in a way that opponent might get hold of it is discouraged.
In a minimal form, the shaping component I added to the can be written as

$$r_t^{\text{shape}} = w_c\, c_t - w_{\text{opp}}\,\tilde{t}_t + \mathbb{I}_{\text{danger}}(s_t)\, w_s\, v_t, \tag{3}$$

where c_t is the closeness proxy, $\tilde{t}_t$ is the opponent touch/possession proxy, and v_t is a speed-to-puck term that is *only* active when agent it under 'danger' (via $\mathbb{I}_{\text{danger}}$) and it encourages the agent to move towards

the puck in higher speed. Potential-based shaping and action L2 regularization are also part of the full reward design I used in my final training, but I leave their detailed formulation to the next section, since my teammate implemented those components. The comparison plots in Fig. 1 shows that both my speed and opponent puck possession proxies assisted in my agent in a significantly higher TrueSkill rating among the pool of my trained agents.

2.1.3 Environment Modification

The original environment initialized `Mode.NORMAL` with a fixed opponent vertical position at mid-ice ($y = H/2$) and only a small puck perturbation ($y = H/2 + \mathcal{U}(-H/8,\, H/8)$). Based on my visualizations, this limited early-game diversity. To fix this, I randomized the opponent's spawn to $y = H/2 + \mathcal{U}(-H/4,\, H/4)$ and widened the puck offset to $y = H/2 + \mathcal{U}(-H/3,\, H/3)$, which trained my agent on a broader set of opponent approach angles and trajectories and allowed a more general defensive policy. The effect of this modification is clear as the environment comparison plot in Fig. 1 shows the modified environment achieves markedly stronger TrueSkill-based performance than the original setup among my trained agents.

2.1.4 Self-Play: Prioritized Fictitious Self-Play

Prioritized Fictitious Self-Play (PFSP) is an opponent-sampling curriculum for self-play: instead of training against a uniform mix of past opponents, we prioritize opponents that are "close" to the current policy, because they produce the most informative learning signal. With a pool S and estimated win-rate $p_{\text{win}}(\pi, s)$, a common PFSP weighting is

$$w(s) = \big(p_{\text{win}}(\pi, s)\,(1 - p_{\text{win}}(\pi, s))\big)^{\alpha}, \qquad P_{\text{PFSP}}(s) = \frac{w(s)}{\sum_{s' \in S} w(s')},$$

which peaks at $p_{\text{win}} \approx 0.5$ and downweights opponents that are too easy or too hard. This accelerates progress by focusing training on mistakes that still matter, but pure PFSP can become rather myopic. Based on my observations, noisy win-rate estimates and uneven sampling caused overfitting to a specific sections of the pool and missed some rarer but important opponents. This is why I implemented one more method explained below.

2.1.5 Self-Play: Upper Confidence Bound

To address the PFSP myopia, I added a UCB exploration stage after my agent went through a phase of training by PFSP. Tracking matches n_s for each opponent and $N = \sum_s n_s$, UCB uses the formula $b(s) = c\sqrt{\frac{\ln N}{n_s + \varepsilon}}$, and samples using a combined score (e.g., $P(s) \propto \exp(\tau(\log w(s) + b(s)))$. This cause made the opponent selection to balance the "currently useful" opponents from PFSP with less tested opponents.The UCB term revisits under-sampled opponents and reduces blind spots, while my PFSP method concentrates learning on competitive matchups.

This improved the agent's generalization during training. After the training phase under UCB finished, I then trained the agent through one phase of PSRO league training for maximal robustness. The implementation of PSRO was done by Kourosh, which worked well on top of my PFSP+UCB setup, hence using this combination for our experiments on both TQC and SAC. As shown in Fig. 4, the agents achieved continuous improvements in their TrueSkill-based leaderboard (described in section 3) through various training phases (first PFSP, then UCB, and finally PSRO).

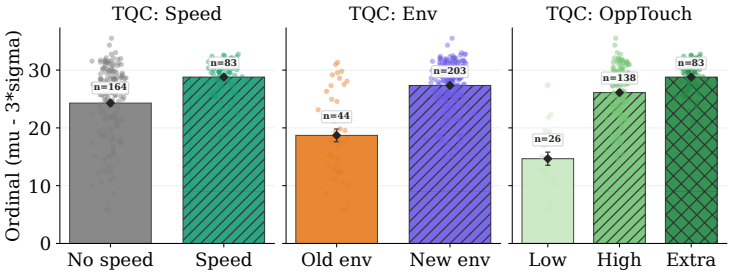

Figure 1: TrueSkill score summary for TQC across configuration changes. Bars show the mean Ordinal score ($\mu - 3\sigma$) within each bucket, with error bars indicating the standard error of the mean. Panels compare (left) speed-reward on/off, (middle) old vs new environment, and (right) opponent-touch-weight level.

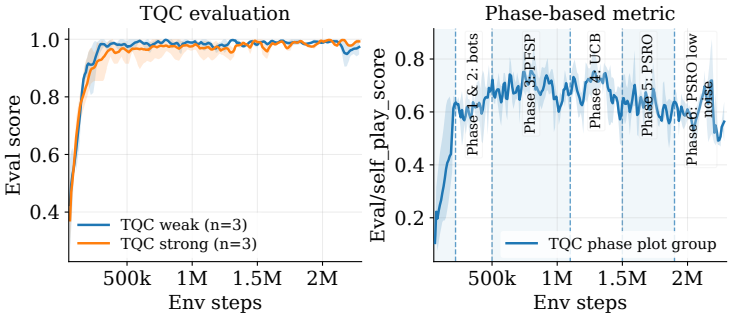

Figure 2: TQC final training. **(a)** Mean evaluation score versus environment steps across three seeds. **(b)** Mean self-play score across the 6-phase curriculum. Vertical lines and shading indicate phase transitions.

2.1.6 Experiments and Results

My training follows a 6-phase schedule. The first phase is a defense-only and training-only training using the environment's built in system before we switch to a phase of training against 75% strong agents and 25% weak agents. Then I add the self-learning opponents from stage three, where I transition from PFSP algorithm in stage three to UCB in stage four and PSRO in stage five. There is then a final stage of no noise training where I continue training using PSRO for a limited period. The TQC agent was trained during these stages for a total of 30,000 episodes. The training of my final agent could be seen in Fig. 4.

In addition to self-play against recent snapshots, the training also includes external agents: a fixed pool of previously trained policies produced with our own codebase and filtered to be high-performing. These serve as extra-strong opponents in the sampling pool to stress-test and harden new agents beyond the built-in heuristic bots. Our training successfully achieved a high near 100% win rate on the weak and strong bots of the environment after around 500,000 steps, as shown in Fig. 2, and the focus after that was on slowly increasing the opponent pool and the strength of the self-play/external agents.

The reward function was such that a penalty for giving the puck to the opponent. This penalty is scheduled to increase over training per phase. We refer to an increase from 0.2 to 0.5 as the *weak* effect and an increase from 0.3 to 0.8 as the *strong* effect in our experiments. Over phases, the opponent mixture is progressively hardened: the proportion of self-play and external agents increases over time, while the proportions of weak and strong heuristic agents decrease.

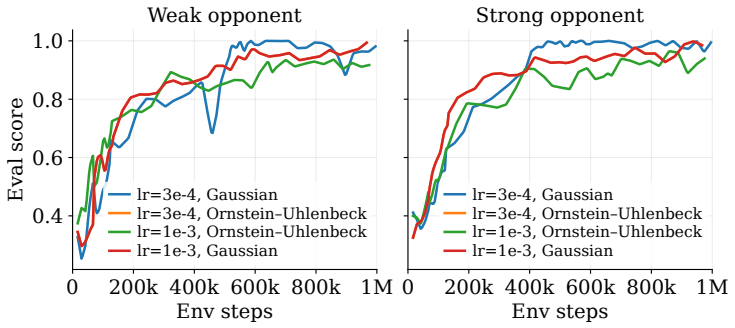

Figure 3: TQC ablation study. The noise variant curves represent the comparison between OU Noise and standard Gaussian noise across different learning rates, evaluated against weak and strong opponents.

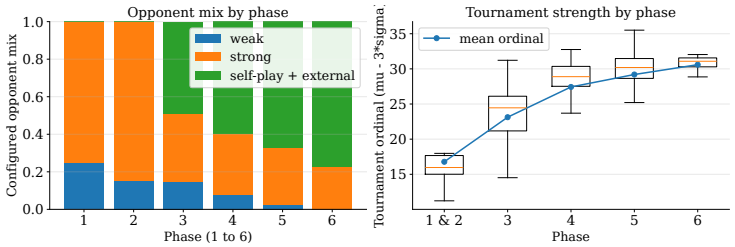

Figure 4: Opponent mix by phase and tournament average $\mu - 3 * \sigma$ scores by phase for TQC.

2.2 Soft Actor-Critic (SAC)

As the second algorithm for this project, I (**Kourosh Sharifi**) implemented Soft Actor-Critic (SAC) [4]. SAC is an off-policy actor-critic method based on the maximum entropy reinforcement learning framework. I chose this algorithm due to its stability and efficient exploration, which are important in high-dimensional continuous action spaces, such as the hockey environment.

2.2.1 Implementation

In my approach, the agent learns to maximize both the expected return and the policy entropy. The objective function is defined as:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t, a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))] \tag{4}$$

where α is the temperature parameter. To avoid the need for manual tuning of α, I implemented an automated entropy adjustment mechanism that constrains the entropy to a target value $\bar{\mathcal{H}}$ (typically $-\dim(\mathcal{A})$) by minimizing the dual objective: $J(\alpha) = \mathbb{E}_{a_t\sim\pi_t}[-\alpha \log \pi_t(a_t|s_t) - \alpha\bar{\mathcal{H}}]$
My actor network architecture consists of a 2-layer MLP with 256 hidden units each, outputting the parameters of a squashed Gaussian distribution. The actions are sampled and transformed via $a_t = \tanh(u_t)$ to respect the environment's $[-1, 1]$ bounds. To mitigate the overestimation of Q-values, I utilize clipped double-Q learning with two critic networks. The target value for the critic update is:

$$y_t = r_t + \gamma(\min_{i=1,2} Q^{\text{targ}}_{\theta_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1}|s_{t+1})) \tag{5}$$

As shown in Fig. 5(a), this method enables stable convergence against weak and strong opponents.

2.2.2 Reward Shaping and Regularization

To fix the sparsity issue of the goal-based reward, I developed my own reward shaping system. While Abtin focused on proximity and possession proxies, I implemented Potential-Based Reward Shaping (PBRS) [9] and kinematic regularization to stabilize the trajectories.

Potential Shaping and Progress Weight. I defined a potential function $\Phi(s)$ based on the Euclidean distance between the puck and the opponent's goal. The shaped reward becomes:

$$r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t) \tag{6}$$

Additionally, I implemented a *progress weight* (w_p) that rewards the agent for the change in distance to the puck between time steps. This makes sure that our agent maintains high-velocity approaches rather than just proximity. The impact of these shaping components is visible in the left panel of Fig. 7, where my agents with active progress and potential rewards achieved higher ordinal scores.

Action L_2 Regularization. To prevent the "jittery" movement common in high-frequency RL games, I added an L_2 penalty to the reward function:

$$r_{\text{final}} = r'_t - w_{L2}\|a_t\|_2^2 \tag{7}$$

In my experiments, a weight of $w_{L2} = 0.0005$ was optimal for allowing smooth, efficient control without sacrificing the power of the shots.

2.2.3 League Training and PSRO

While Abtin's agents used PFSP and UCB for self-play, I made a *League Training* framework based on Policy-Space Response Oracles (PSRO) [8] for the final stages of training. This framework treats self-play as a *meta-game*.

Replicator Dynamics. In our own internal league, a pool of snapshots and compute a meta-strategy π_{meta} were maintained using Replicator Dynamics. This allows the current agent to train against a Nash Equilibrium of past versions, preventing the cycling behavior where an agent forgets how to beat an old opponent. The population distribution x evolves according to:

$$\dot{x}_i = x_i[(Ax)_i - x^T Ax] \tag{8}$$

where A is the payoff matrix calculated through round-robin matches in my league. As shown in Fig. 5(b) and Fig. 8, the transition into the PSRO meta-sampling phase corresponds to an increase in the agent's Ordinal score, as it is forced to generalize against a mix of various opponents with different playing styles.

2.2.4 Results and Experiments

While standard SAC uses uncorrelated Gaussian noise, I checked the effects of temporally correlated *Pink Noise* (or $1/f$ noise) [2]. In an environment like hockey, which requires multi-step action sequences, Pink noise provides more coherent exploration compared to standard white noise, however the difference is not that noticeable. As shown in the ablation study illustrated in Fig. 6, the Pink noise variant (labeled as "noise variant") consistently outperformed standard exploration in the early curriculum phases.

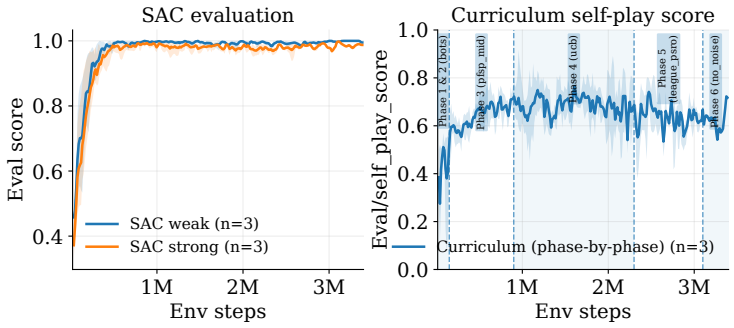

Figure 5: SAC final training. **(a)** Mean evaluation score versus environment steps across three seeds. **(b)** Mean self-play score across the 6-phase curriculum. Vertical lines and shading indicate phase transitions.

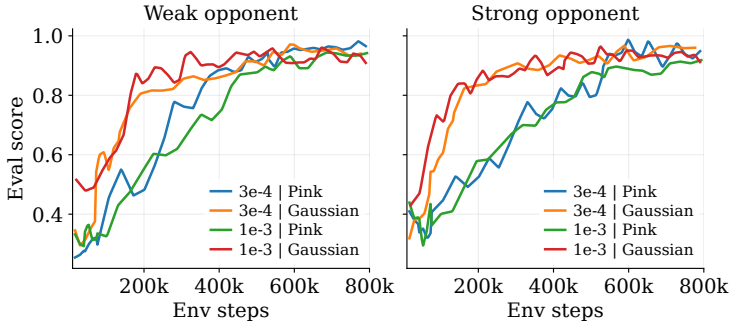

Figure 6: SAC ablation study. The "noise" variant curves represent the comparison between Pink Noise and standard Gaussian noise across different learning rates, evaluated against weak and strong opponents.

3 Internal Leaderboard and Tournament

To rigorously assess the agent's skill, a round-Robin tournament was conducted among the best models we had trained in the last month. The rating of each agent is calculated using the OpenSkill library [6], based on the Plackett-Luce model. The final tournament we used to select our best models included all our TQC and SAC variants trained under different reward configurations that were able to receive a total score of over 0.85 (a weighted average between the self-play, weak, and strong opponents), effectively taking all our past agents that had over a 95% results over the weak and agents opponents into account. This resulted in a total of 500 of our agents being ranked based on round-robin's method.
In addition to round-robin's rankings based on the $\mu - 3 * \sigma$ scores, we further took into account the robustness of our agents by identifying "hard counters" count of each agent (opponents that made one agent's win rate to go below 25%). We then used these results to finalize which agents we want to uload to the course's final tournament.

4 Discussion

We observed that the performance depended on building a stable training pipeline. A detailed phase-by-phase curriculum (shooting, defense, then normal play) reduced early collapse into passive behaviors and made learning more reliable once we increased opponent difficulty. In later stages, improvements were driven primarily by generalization to diverse opponents rather than further gains against the built-in weak/strong bots.

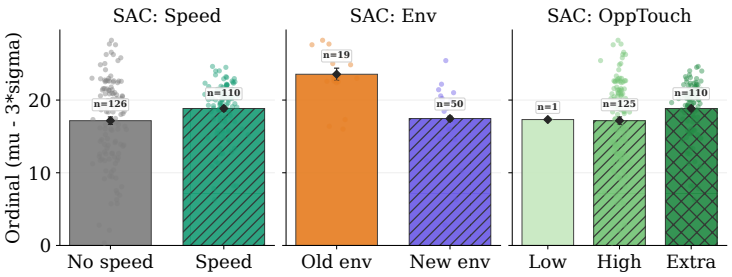

Figure 7: Leaderboard summary for SAC. The left panel demonstrates the performance delta when applying my potential-based shaping and progress weights ("speed-reward").

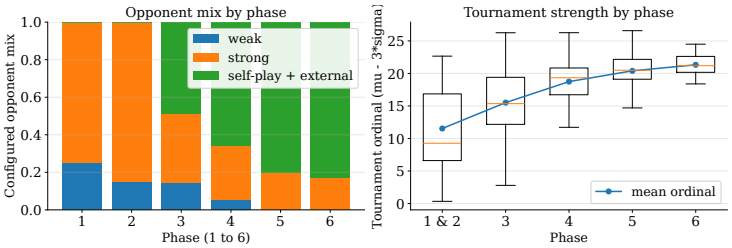

Figure 8: Opponent-mix and tournament strength for SAC. The transition from Phase 4 to Phase 6 (PSRO/Meta) shows the refinement of the agent's Ordinal score as the meta-strategy converges.

We also observed that opponent diversity is essential for training a generalized agent. Our PFSP method improved sample efficiency by focusing training on opponents near the agent's current skill level, but is also missed some less seen agents when win-rate estimates were noisy or when rare opponents are under-sampled. Adding exploration-oriented sampling using UCB over PFSP us helped to expose blind spots, and the PSRO sampling in our final phases reduced cycling by training against a distribution of historical strategies rather than a single recent snapshot.

Reward shaping also accelerated our learning but required careful balance. Proximity and possession proxies helped with sparse rewards and encouraged puck engagement, while potential-based shaping and mild action regularization improved our agent's long-term progress and control stability. However, over-weighting proxies sometimes led us to reward hacking sometimes (e.g., moving near the puck without converting to goals or over-reliance to wall-shots). Increasing diversity in initial states further improved our robustness by broadening the state distribution encountered during training. This was especially so after we introduced the external agents that were trained on various algorithms, reward combinations, and sampling strategies to our training.

Our internal tournament (OpenSkill Plackett–Luce ordinal scoring) was especially crucial in this project because bot win rates often converged to close to 100% and thus failed to reflect competitive strength. By the end, we observed that our TQC agents achieved better ordinal scores than SAC agents in our tournament of roughly 500 agents. We believe this is largely explained by compute. As many TQC agents were trained for more than two days, while SAC agents were typically trained for at most one day, making training time a major confound. We predict that a fair comparison of similar training times would allow SAC to achieve results comparable to our TQC algorithm. One final run of our internal tournament CSV on our 500 agents file is included in our final code submission of the course.

References

[1] X. Chen, C. Wang, Z. Zhou, and K. Ross. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations (ICLR)*, 2021.

[2] O. Eberhard, J. Hollenstein, M. Mutny, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations*, 2023.

[3] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning*. PMLR, 2018.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.

[5] T. Hiraoka, T. Imagawa, T. Hashimoto, T. Onishi, and Y. Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2022.

[6] V. Joshy. Openskill: A faster asymmetric multi-team, multiplayer rating system. *Journal of Open Source Software*, 9(93):5901, 2024.

[7] A. Kuznetsov, P. Shvechikov, A. Grishin, and D. Kuzovkin. Controlling overestimation bias with truncated quantile critics. *International Conference on Machine Learning*, 2020.

[8] M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Babuschkin, D. Lessard, and D. Silver. A generalized training approach for multiagent learning. *Advances in Neural Information Processing Systems*, 2017.

[9] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287. Morgan Kaufmann, 1999.

[10] M. Towers et al. Gymnasium, 2023.